Use Cases

Project: Aura Promptly
Version: 1.1
Date: 2026-05-14
Status: Baselined

Change Log

Version	Date	Changes
1.1	2026-05-14	UC-01: Aurora pgvector replaces OpenSearch Serverless; two-layer observability model. UC-02: demo task updated to Research Assistant governance query. UC-03: GCP auth updated to direct STS federation (no Cognito).
1.0	2026-05-14	Initial baseline

Use Case Summary

ID	Name	Actor	Priority
UC-01	RAG Chatbot (Azure Spoke)	Azure application developer	MUST
UC-02	Agentic Query (AWS Spoke)	AWS application developer	MUST
UC-03	Document Summarisation (GCP Spoke)	GCP application developer	MUST
UC-04	Central Operator: View Spend & Usage	Central AI Governance team	MUST
UC-05	Central Operator: Configure Guardrails	Central AI Governance team	MUST
UC-06	Central Operator: Add/Remove Model Route	Central AI Governance team	SHOULD
UC-07	Platform Engineer: Deploy Hub	Platform Engineer	MUST
UC-08	Platform Engineer: Destroy & Recreate Infrastructure	Platform Engineer	MUST
UC-09	Platform Engineer: Add New Spoke Application	Platform Engineer	SHOULD

ID	Name	Actor	Priority
UC-10	Knowledge Base: Ingest New Documents	Platform Engineer / Governance team	MUST

UC-01 — RAG Chatbot (Azure Spoke)

Actor: Developer / end user of an application hosted on Azure
Goal: Ask a natural-language question and receive a grounded answer from the centralised Knowledge Base

Preconditions

- Azure sample application is deployed (Azure Container Apps)
- Hub endpoint URL and API key are configured in the app
- Bedrock Knowledge Base has been ingested with at least one document

Main Flow

```
1. User types a question into the Streamlit UI
2. Azure app sends POST /chat/completions to the Hub API Gateway
   - Header: x-api-key: <team-api-key>
   - Body: { model: "bedrock-haiku-kb", messages: [...] }
3. API Gateway validates the API key via Cognito / API Gateway usage plan
4. API Gateway forwards the request to LiteLLM Proxy (ECS Fargate)
5. LiteLLM:
   a. Checks team spend budget → within limit, proceed
   b. Routes to Bedrock Knowledge Base retrieve-and-generate endpoint
   c. Applies Bedrock Guardrails check
6. Bedrock retrieves relevant chunks via Knowledge Base (Aurora pgvector,
HNSW index)
7. Bedrock passes retrieved context + question to Claude Haiku 3
8. Response flows back through LiteLLM → API Gateway → Azure app → UI
9. LiteLLM logs: tokens used, latency, team ID, model, guardrail result
10. Log shipped to CloudWatch (Layer 1 — always-on); OTel Collector
forwards to Grafana Cloud (Layer 2 — optional, enable_grafana_export=true)
```

Alternate Flows

- **5a fails (budget exceeded):** LiteLLM returns HTTP 429 with message "Team budget exceeded"
- **5c fails (guardrail violation):** LiteLLM returns HTTP 400 with guardrail reason
- **Bedrock unavailable:** LiteLLM falls back to OpenAI GPT-4o-mini (no KB grounding, plain LLM response)

Postconditions

- User sees a grounded answer with source references
- Usage is recorded in LiteLLM spend dashboard

- Log entry visible in Grafana within 60 seconds

UC-02 — Agentic Query (AWS Spoke)

Actor: Developer / end user of an application hosted in an AWS spoke account
Goal: Submit a multi-step task to the Bedrock Agent via the hub and receive a completed result

Preconditions

- AWS spoke sample application is deployed (Lambda + API Gateway or ECS)
- Spoke app authenticates to hub using IAM role + STS AssumeRole (cross-account)
- Bedrock Agent and Action Groups are deployed in hub account

Main Flow

```
1. User submits a governance research query (e.g. "Which AI use cases
require mandatory human oversight under our policy?")
2. AWS spoke app calls Hub API Gateway using AWS Signature V4 (IAM auth)
3. API Gateway validates IAM credentials (cross-account role)
4. LiteLLM routes to Bedrock Agent invoke endpoint
5. Bedrock Agent:
   a. Plans the task (ReAct loop)
   b. Calls Action Group Lambda (e.g. fetch-document Lambda)
   c. Receives tool result, continues planning
   d. Returns final response
6. Response flows back to spoke app → UI
7. Full agent trace is logged (each ReAct step)
```

Alternate Flows

- **Action Group Lambda times out:** Agent returns partial result with error annotation
- **IAM cross-account role misconfigured:** API Gateway returns 403; error logged

---

UC-03 — Document Summarisation (GCP Spoke)

Actor: Developer / end user of an application hosted on GCP
Goal: Submit a document (text) for summarisation and receive a concise summary

Preconditions

- GCP spoke sample application is deployed (Cloud Run)
- Spoke app authenticates via GCP Workload Identity OIDC → AWS STS AssumeRoleWithWebIdentity → IAM Sig V4 (direct federation, no Cognito)

Main Flow

```
1. User uploads or pastes a text document into the Streamlit/Gradio UI
2. GCP app uses Workload Identity metadata server to obtain OIDC token;
calls AWS STS AssumeRoleWithWebIdentity to get temporary IAM credentials
3. GCP app sends POST /chat/completions to Hub API Gateway, request signed
with IAM Sig V4
4. API Gateway validates IAM Sig V4 signature (IAM authoriser)
5. LiteLLM routes to Bedrock Claude Haiku 3 with a summarisation system
prompt
6. LiteLLM applies guardrails check
7. Response (summary) returned to GCP app → UI
8. Token usage and latency logged
```

UC-04 — Central Operator: View Spend & Usage

Actor: Central AI Governance team member

Goal: Monitor AI consumption, cost and errors across all teams in real time

Flow

```
1. Operator opens LiteLLM Admin UI (authenticated via admin API key)
2. Views: spend per team, spend per model, total requests, error rate
3. Operator opens Grafana dashboard
4. Views: request rate over time, P95 latency, guardrail violation count,
spend trend
5. Operator sets or adjusts a team's monthly budget limit in LiteLLM
config
```

UC-05 — Central Operator: Configure Guardrails

Actor: Central AI Governance team member

Goal: Update the content guardrails applied to all model calls without redeploying the application

Flow

```
1. Operator updates Bedrock Guardrails configuration (via AWS Console or
Terraform)
2. Operator updates LiteLLM callback filter configuration (via LiteLLM
Admin API or config file)
3. Changes take effect on the next inference request (no restart required
for LiteLLM callbacks)
4. Operator sends a test request that should be blocked → verifies 400
response
5. Violation is visible in Grafana dashboard
```

UC-06 — Central Operator: Add/Remove Model Route

Actor: Central AI Governance team member / Platform Engineer
Goal: Add a new model (e.g. Claude Sonnet) or disable a model route without downtime

Flow

```
1. Engineer updates LiteLLM config YAML: adds new model entry or sets
model status to "disabled"
2. LiteLLM hot-reload picks up the change (or ECS task is restarted if
hot-reload not triggered)
3. Engineer tests the new route via a curl command
4. Usage of new model appears in spend dashboard
```

UC-07 — Platform Engineer: Deploy Hub

Actor: Platform Engineer (novice-friendly)
Goal: Deploy the entire hub infrastructure from scratch

Flow

```
1. Clone repository
2. Configure Terraform Cloud workspace credentials
3. Run: cd infra/hub && terraform init
4. Run: infracost breakdown --path . (review cost estimate)
5. Run: terraform apply (approve prompt)
6. Terraform outputs: API Gateway URL, LiteLLM Admin URL, Grafana
dashboard URL
7. Run post-deploy verification script (tests all endpoints)
```

Target time: < 30 minutes end-to-end for an experienced operator, < 90 minutes for a novice following the runbook.

UC-08 — Platform Engineer: Destroy & Recreate Infrastructure

Actor: Platform Engineer
Goal: Tear down all infrastructure to zero cost when not actively working

Destroy Flow

```
1. Trigger GitHub Actions workflow: "Destroy Infrastructure"
(workflow_dispatch)
   OR run locally: terraform destroy -auto-approve
2. Terraform destroys all resources in reverse dependency order
```

```
3. Terraform Cloud state updated to empty
4. Cost: $0/day from this point
```

Recreate Flow

```
1. Trigger GitHub Actions workflow: "Deploy Infrastructure"
   OR run locally: terraform apply -auto-approve
2. All resources recreated in ~20 minutes
3. Knowledge Base re-sync triggered automatically post-deploy
```

UC-09 — Platform Engineer: Add New Spoke Application

Actor: Platform Engineer
Goal: Onboard a fourth application team to the hub

Flow

```
1. Create new Terraform workspace: infra/spoke-<name>
2. Copy spoke template module, update variables (team name, cloud
provider, auth method)
3. Run terraform apply
4. Generate API key for new team in LiteLLM (via Admin API or config)
5. Set team budget in LiteLLM
6. Provide team with: API Gateway URL, API key, example code snippet
```

UC-10 — Knowledge Base: Ingest New Documents

Actor: Platform Engineer / Governance team
Goal: Add new documents to the RAG Knowledge Base

Flow (PoC — web content via S3)

```
1. Download or convert web content to text/PDF
2. Upload file(s) to S3 bucket: s3://aura-promptly-kb-<account-id>/
   (via AWS CLI, Console, or GitHub Actions workflow)
3. Trigger Knowledge Base sync:
   a. Via GitHub Actions: workflow_dispatch "Sync Knowledge Base"
   b. Via AWS CLI: aws bedrock-agent start-ingestion-job ...
4. Sync completes (typically 1-5 minutes for small documents)
5. New content is immediately available to RAG queries
```

Future Flow (Confluence / SharePoint — documented stub)

```
1. Configure Bedrock Knowledge Base connector (Confluence or SharePoint
type)
2. Provide OAuth2 credentials (stored in Secrets Manager)
3. Set sync schedule (daily or on-demand)
4. Content indexed automatically
```